

Contents

1	A First Analysis of the FDR String Matcher	2
1.1	Pseudocode	2
1.2	Analysis of a Special Cases	5
1.3	Problem Statement	10
2	Taking Assignment Algorithm into Account	11
2.1	Setting	11
2.2	Assignment Algorithm	11
3	Largest Concatenation-Component in Regular Expressions	12
3.1	Recurrences	12
3.2	The yield recurrence	13
3.2.1	Lower and Upper Bounds of the Expectation	13
3.2.2	Asymptotic of the Expectation	14
4	Asymptotic Analysis of Some Sequences	17
5	Induction	21
5.1	Increasingness	21
5.1.1	Analysis	21
5.1.2	Proof	22
5.2	Asymptotic	23
5.2.1	Analysis	24
6	Monte-Carlo Simulation for Checking Conjectures	25
6.1	Background	25
6.1.1	Empirical Distributions	25
6.1.2	Random Distributions	26
7	Appendix	27

A First Analysis of the FDR String Matcher

February 18th, 2026

A First Analysis of the FDR String Matcher

We extract a pseudocode of a simplified version of FDR for later analysis. A Python implementation so that we can inject instruction counter is also given in the repository. We begin with the pseudocode. Then we solve special cases and try to formulate the problem in a more general way.

1.1 Pseudocode

The algorithm extends Shift-Or algorithm for multiple patterns. The conventions are as follows. The number of bits of a character is 8. There are 8 buckets $B_0, \dots, B_7$. Each register is of size 128 bits. We access the bit indices in a register and the character positions in a string *right to left*.

The compilation phase takes a list of patterns, put them into buckets and builds a mask for each character. The mask for a character c is a bitmask of size 128 where the left-most 64 bits are set to 0. In the right-most 64 bits, the $(8i + j)$ -th bit ($0 \leq i, j \leq 7$) is set to 0 if either

- (i) c appears in position i (indexing from the right) of a pattern in bucket B_j , or
- (ii) bucket B_j is not empty and patterns in B_j have length less than or equal to i . In this case it is called a *padding bit*.

Otherwise, the bit is set to 1.

For example, consider the buckets $[[ab, bc], [abc], [], [], [], [], []]$. Then the mask for each character is as follows. The padding bits are in bold.

$$\begin{aligned} M[a] &= \underbrace{00 \dots 00}_{64} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111110} \text{ 11111111} \\ M[b] &= \underbrace{00 \dots 00}_{64} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111110} \text{ 11111100} \text{ 11111110} \\ M[c] &= \underbrace{00 \dots 00}_{64} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111110} \text{ 11111111} \text{ 11111101} \\ M[x] &= \underbrace{00 \dots 00}_{64} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111}\mathbf{00} \text{ 11111110} \text{ 11111111} \text{ 11111111}. \end{aligned}$$

In the masks above, x is any character rather than a, b, c .

The execution phase takes a text and the data structure built in the compilation phase. It maintains a state mask R . It is better to interpret the bits in R as follows. Each iteration processes 8 bytes. At iteration $ITER$, the $(8i + j)$ bit ($0 \leq i \leq 15, 0 \leq j \leq 7$) is 0 means that there is currently *no contradiction* that there is a match of a pattern in bucket B_j ending at position $8 \times ITER + i$. If bucket B_j has patterns of length ℓ_j , then there cannot be a match in

this bucket at position less than $\ell_j - 1$. Therefore, we initialize the state mask to all 0's, except for $8i + j$ where $0 \leq i < \ell_j - 1$. After an iteration, we update $R := R \gg 64$, continuing matching the next 8 bytes. By that way, we ensure that at each position, the prefix of length at most 8 is considered.

Coming back to the example, the initial state mask is

$$R = \underbrace{00 \dots 00}_{104} \ 11111100 \ 11111110 \ 11111111.$$

Let us execution on the text *bab*c. Since there are only 4 character in the text, there is only one iteration. We give details about scanning each character as follows, where we consider the right-most 4 bytes. The newly bit set to 1 after each character is scanned is underlined.

Scan	Update	Result
<i>b</i>	$R \leftarrow R (M[b] \ll 0)$	... 11111100 111111 <u>1</u> 0 11111110 11111111
<i>a</i>	$R \leftarrow R (M[a] \ll 1)$	... 11111100 11111110 1111111 <u>1</u> 11111111
<i>b</i>	$R \leftarrow R (M[b] \ll 2)$	... 11111100 11111110 11111110 11111111
<i>c</i>	$R \leftarrow R (M[c] \ll 3)$	... 1111110 <u>1</u> 11111110 11111110 11111111

Now we report that there is a match at bucket B_0 that ends at position 1, a match at bucket B_1 that ends at position 2 and a match at bucket B_0 that ends at position 3. Since we already know the length of the patterns in each bucket, we can retrieve the beginning position of those potential matches and verify them by exact string comparison. In this case, those three potential matches are indeed matches. We give the pseudocode as in Algorithm 1 and Algorithm 2.

Algorithm 1 Compilation phase of the FDR string matcher

```

1: Assign each pattern to a bucket  $B_1, \dots, B_b$ .
2:  $M \leftarrow$  a dictionary mapping each character to a mask.
3: for  $c = 00000000$  to  $11111111$  do
4:    $M[c] \leftarrow \underbrace{00 \dots 00}_{64} \underbrace{11 \dots 11}_{64}$  ▷ Initialize.
5:   for  $j = 0$  to  $7$  and  $B_j$  is not empty do
6:      $\ell_j \leftarrow$  length of the longest pattern in bucket  $B_j$ .
7:     for  $i = \ell_j$  to  $7$  do
8:        $M[c][8i + j] \leftarrow 0$  ▷ Set padding bits to 0.
9:     endfor
10:  endfor
11: endfor
12: for  $j = 0$  to  $7$  do
13:   for each pattern  $x$  in bucket  $B_j$  do
14:     for  $i = 0$  to  $\text{length}(x) - 1$  do
15:        $M[x[i]][8i + j] \leftarrow 0$ 
16:     endfor
17:   endfor
18: endfor

```

Algorithm 2 Execution phase of the FDR string matcher

```
1:  $R \leftarrow$  a bitmask of size 128 where the  $(8i + j)$ -th bit is 0 if  $i \geq \ell_j - 1$  and 1 otherwise.
2:  $T \leftarrow$  the text to be scanned.
3: for ITER = 0 to  $\lfloor \frac{\text{length}(T)}{8} \rfloor - 1$  do
4:    $V \leftarrow T[8 \times \text{ITER} \dots 8 \times \text{ITER} + 7]$ 
5:   for  $j = 0$  to 7 do
6:      $R \leftarrow R[(M[V[j]] \ll j)]$ 
7:   endfor
8:   Report potential matches at positions where the corresponding bits in  $R$  are 0. Verify
   those potential matches by exact string comparison.
9:    $R \leftarrow R \gg 64$ 
10: endfor
```

Coming back to the buckets in the example. The current algorithm cannot give contradiction to the text ac at position 1 for bucket B_0 . *Super-characters* are introduced. Let s be the number of bits in a super-character. For example, instead of assigning $M[a]$ as in the first step of the compilation phase, we build a mask for the super-character

$$(b[0 \dots s - 8] \ll 8) | a.$$

If a character is at the end of a patterns, for example b in the pattern ab , then we assign the mask for

$$0 \ll 8 | b.$$

But now, the algorithms may *fail* to report the match of ab at bucket B_0 that ends at position 2 for the given text $abac$. The reason is that when scanning the second b , it looks and ors the mask for the super-character $(b[0 \dots s - 8] \ll 8) | c$ instead of the mask for $0 \ll 8 | b$. The idea is to set the bit(s) that may introduce a wrong contradiction to 0 in the mask for the super-character beforehand. More precisely, if a character c appears at position 0 of a pattern (that is, a bit confusing, at the end!) in bucket B_j , then we set the bit at position j to 0 in the mask for the super-character

$$(x[0 \dots s - 8] \ll 8) | c$$

for *every* character x . This can be done more efficiently, as in fact there are only 2^{s-8} such super-characters. The corrected compilation pseudocode is given in Algorithm 3. The changes or additions are highlighted in blue.

Algorithm 3 Compilation phase of the FDR string matcher

```
1: Assign each pattern to a bucket  $B_1, \dots, B_b$ .
2:  $M \leftarrow$  a dictionary mapping each character to a mask.
3:  $s \leftarrow$  the number of bits in a super-character.
4: for  $c = \underbrace{00 \dots 00}_s$  to  $\underbrace{11 \dots 11}_s$  do
5:    $M[c] \leftarrow \underbrace{00 \dots 00}_{64} \underbrace{11 \dots 11}_{64}$  ▷ Initialize.
6:   for  $j = 0$  to 7 and  $B_j$  is not empty do
7:      $\ell_j \leftarrow$  length of the longest pattern in bucket  $B_j$ .
8:     for  $i = \ell_j$  to 7 do
9:        $M[c][8i + j] \leftarrow 0$  ▷ Set padding bits to 0.
10:    endfor
11:  endfor
12: endfor
13: for  $j = 0$  to 7 do
14:   for each pattern  $x$  in bucket  $B_j$  do
15:      $f \leftarrow x[0]$  ▷ Handle the end character.
16:     for  $y = \underbrace{0 \dots 0}_{s-8}$  to  $\underbrace{1 \dots 1}_{s-8}$  do
17:        $M[(y \ll 8) | f][8j] \leftarrow 0$ 
18:     endfor
19:     for  $i = 1$  to  $\text{length}(x) - 1$  do
20:        $M[x[i]][8i + j] \leftarrow 0$ 
21:     endfor
22:   endfor
23: endfor
```

We comment on the original implementation. In the execution phase, the original implementation lets us traverse j from 0 to 7 with skipping. This parameter is called *stride*. For example, if the stride is 2, then we traverse j from 0 to 7 with step 2. This is because each step of j , we have to query the mask dictionary, which is not in constant time. The trade-off is that there are more positives to verify.

The related paper [4] claims that the algorithm outperforms Aho-Corasick (AC). But AC is already linear in the length of the text, asymptotically. So we may have to analyze very closely to the original implementation to understand the reason. In fact, there are some optimizations in the original implementation that are not reflected in the pseudocode. They leverage SIMD everywhere possible. For example, they encode the mask table in byte codes and access them using (aligned) addresses. Hence I would like to firstly understand why the complexity of FDR is not linear in the number of patterns. I begin at special cases to gain experience, then gradually study more general distributions of patterns and texts, and add super-characters, strides and hashing.

1.2 Analysis of a Special Cases

Let there be only one bucket and all patterns have the same length 8. This gives an intuition that the strength of the algorithms lies not mainly in parallelism but in the fact that false positives are rare. SIMD helps because there are long registers.

It is clear that the complexity of the compilation phase is $\mathcal{O}(p)$, since the number of characters to build masks are fixed.

In the execution phase, each position in the state mask R has to be or-ed at most 8 times, while 8 positions are updated at the same time. Hence the complexity is roughly given by

$$\begin{aligned}
C &= \mathcal{O}(n) \times [\text{cost of each iteration}] \\
&= \mathcal{O}(n) \times \left([\text{cost of ors}] \right. \\
&\quad + \mathbb{P}(\text{no positive}) \times 0 \\
&\quad + \mathbb{P}(\text{positive but not matched}) \times [\text{cost of rejection}] \\
&\quad \left. + \mathbb{P}(\text{positive and matched}) \times [\text{cost of confirmation}] \right).
\end{aligned}$$

If hashing is used, then the cost of rejection would be less than the cost of confirmation. In our case, since we are doing exact string comparison, the cost of rejection and confirmation are in $\mathcal{O}(p)$.

Let $X_1, \dots, X_p \stackrel{iid}{\sim} \mathcal{U}(\{0, 1\}^{64})$ be the patterns, uniformly distributed in the set of binary strings of length 64. Let $Y \sim \mathcal{U}(\{0, 1\}^8)$ be the text at an iteration. This implies that every character is drawn independently and uniformly at random. Suppose that there is only one bucket B_0 and all patterns are in this bucket. Let A be the event that there is a match ending at a position and B be the event that there is a positive. Then $A \subset B$. Let D be the event that all patterns are distinct. We rewrite the complexity as follows.

$$\begin{aligned}
C &= \mathcal{O}(n)(1 + \mathbb{P}(B^c \mid D) \times 0 + \mathcal{O}(p) \times (\mathbb{P}(A, B \mid D) + \mathbb{P}(A^c, B \mid D))) \\
&= \mathcal{O}(n)(1 + \mathcal{O}(p) \times (\mathbb{P}(A \mid D) + \mathbb{P}(B \mid D) - \mathbb{P}(A \mid D))) \\
&= \mathcal{O}(n)(1 + \mathcal{O}(p) \times \mathbb{P}(B \mid D)).
\end{aligned}$$

We check that $\mathbb{P}(B^c \mid D) + \mathbb{P}(A, B \mid D) + \mathbb{P}(A^c, B \mid D) = 1$, so the formulation should be correct. The algorithm is efficient when $\mathbb{P}(B \mid D)$ is negligible. In the first time, I tried to compute $\mathbb{P}(B \mid D)$ directly. Then I realized that it is not much different if we do not condition on D . Let me give this computation later. Maybe it can be used for more general cases. In fact, $\mathbb{P}(B \mid D)$ is close to $\mathbb{P}(B)$. We have

$$\begin{aligned}
\mathbb{P}(B \mid D) - \mathbb{P}(B) &= \frac{\mathbb{P}(B, D)}{\mathbb{P}(D)} - \mathbb{P}(B, D) - \mathbb{P}(B, D^c) \\
&\leq \mathbb{P}(B, D) \left(\frac{1}{\mathbb{P}(D)} - 1 \right) \\
&\leq \mathbb{P}(D) \left(\frac{1}{\mathbb{P}(D)} - 1 \right) \\
&= 1 - \mathbb{P}(D) = \mathbb{P}(D^c).
\end{aligned}$$

Now we bound $\mathbb{P}(D^c)$. This is the probability that there are two patterns that are the same. Let $N = 2^{64}$. Firstly, we choose 2 among p patterns and set them to be one of the N possible patterns. Then we choose the remaining $p - 2$ patterns from the remaining $N - 1$ possible patterns. There may be some over-counting, so we have

$$\mathbb{P}(D^c) \leq \frac{\binom{p}{2} \times N \times N^{p-2}}{N^p} = \frac{\binom{p}{2}}{N}.$$

This is indeed negligible because N is very large. Now we compute $\mathbb{P}(B)$, the probability that there is a positive. Since Y is uniformly distributed and independent of the patterns, we can fix $Y = 0^{64}$. Let us count the number of possible values of $X_1, \dots, X_p$ such that there is a

positive. There are in total N^p possible values of $X_1, \dots, X_p$. For each position i from 0 to 7, there are $2^8 = 256$ possible characters. But 00000000 is forbidden. Hence there are 255 available possible characters allowed for the patterns. Therefore, there are 255^p possible values of $X_1[8i \dots 8i+7], \dots, X_p[8i \dots 8i+7]$ such that there is no positive. Hence, there are $(N^p - 255^p)^8$ possible values of $X_1, \dots, X_p$ such that there is a positive. Therefore,

$$\mathbb{P}(B) = \frac{(N^p - 255^p)^8}{N^{8p}} = \left(1 - \left(\frac{255}{256}\right)^p\right)^8.$$

When $p \geq 135$, $p\mathbb{P}(B) \geq 0.1$. The algorithm starts to be inefficient. Suppose that p patterns are divided into 8 buckets evenly. Then the new probability of a positive is

$$\mathbb{P}(B) = 1 - \left(1 - \left(1 - \left(\frac{255}{256}\right)^{p/8}\right)^8\right)^8.$$

The new $p\mathbb{P}(B)$ exceeds 0.1 when $p \geq 460$.

Let us use a different approach to compute $\mathbb{P}(B)$, which then can then be applied to compute $\mathbb{P}(B \mid D)$, and the case where super-characters are used. Again, since Y is uniformly distributed and independent of the patterns, we can fix $Y = 0^{64}$. Let $U = \{0, 1\}^{64}$ be the universe. Define the sets

$$S_j = \{x \in U \mid x[8j \dots 8j + 7] = 0^8\}, j \in \{0, \dots, 7\}.$$

Then B is the event that there is at least one pattern in each S_j i.e. the set $\{X_1, \dots, X_p\}$ hits all S_j 's. Then B^c is the event that there is some j such that $\{X_1, \dots, X_p\}$ does not hit S_j .

$$B^c = \bigcup_{j=0}^7 \underbrace{\bigcap_{i=1}^p \{X_i \notin S_j\}}_{E_j}.$$

By inclusion-exclusion,

$$\begin{aligned} \mathbb{P}(B) &= 1 - \mathbb{P}\left(\bigcup_{j=0}^7 E_j\right) \\ &= 1 - \sum_{\emptyset \neq L \subset \{0 \dots 7\}} (-1)^{|L|+1} \mathbb{P}\left(\bigcap_{j \in L} E_j\right) \\ &= 1 + \sum_{\emptyset \neq L \subset \{0 \dots 7\}} (-1)^{|L|} \mathbb{P}\left(\bigcap_{j \in L} \bigcap_{i=1}^p \{X_i \notin S_j\}\right) \\ &= 1 + \sum_{\emptyset \neq L \subset \{0 \dots 7\}} (-1)^{|L|} \mathbb{P}\left(\bigcap_{i=1}^p \left\{X_i \notin \bigcup_{j \in L} S_j\right\}\right) \\ &= 1 + \sum_{\emptyset \neq L \subset \{0 \dots 7\}} (-1)^{|L|} \mathbb{P}\left(X_1 \notin \bigcup_{j \in L} S_j\right)^p \quad (\text{since } X_i \text{'s are iid}). \end{aligned}$$

The events $\{X_1 \notin S_j\}, j \in L$ are in fact independent, since the S_j 's forbid 0^8 at disjoint positions. Let $V_L = (X_1 \notin \bigcup_{j \in L} S_j)$. It will be clear when we calculate $\mathbb{P}(V_L)$. To make V_L happen, we

can only choose $2^8 - 1 = 255$ possible strings for $X_1[8j \dots 8j + 7]$ for each $j \in L$ and $2^8 = 256$ possible strings for each $j \notin L$. Hence,

$$\mathbb{P}(V_L) = \frac{256^{8-|L|} \times 255^{|L|}}{N} = \left(\frac{255}{256}\right)^{|L|}.$$

There is $\binom{8}{\ell}$ subsets of $\{0 \dots 7\}$ of size ℓ . Hence,

$$\mathbb{P}(B) = 1 + \sum_{\ell=1}^8 \binom{8}{\ell} (-1)^\ell \left(\frac{255}{256}\right)^{p\ell} = \left(1 - \left(\frac{255}{256}\right)^p\right)^8.$$

Note that in the computation of $\mathbb{P}(V_L)$, the numerator $256^{8-|L|} \times 255^{|L|} = N (255/256)^{|L|}$ is the number of patterns that can be chosen for each X_i . Conditioning on D means to choose p distinct patterns. Hence,

$$\mathbb{P}(\cap_{j \in L} E_j \mid D) = \frac{\binom{N(255/256)^{|L|}}{p}}{\binom{N}{p}} \approx \left(\frac{255}{256}\right)^{p|L|}.$$

Let us take super-characters into consideration. Recall that we may have to set the bit at position j to 0 in the mask for the super-character

$$(x[0 \dots s-7] \ll 8) \mid c$$

if c appears at the end of a pattern. That is, we only may have to set the bits in the *first* byte, which can be or-ed with the considered position of the text *once*. The other bits of the masks in the super-character case are the same as in the last case. Therefore, now we define

$$S_j^{(s)} = \{x \in U \mid x[j \dots j+7] = 0^8 \text{ and } x[j-8 \dots j+s-15] = 0^{s-8}\}, j \in \{1, \dots, 7\},$$

and similarly

$$E_j^{(s)} = \{X_1 \notin S_j^{(s)}, \dots, X_p \notin S_j^{(s)}\}, j \in \{1, \dots, 7\}.$$

That is, we do not care about position 0, and give an upper bound for $\mathbb{P}(B)$ by only considering the events $E_j^{(s)}$ for $j \in \{1, \dots, 7\}$. Let me first attempt like this, because handling which cases are some masks set to 0 at the first byte seems complicated to me at the moment, since we have to consider characters that appear both at the end and the middle of patterns. Similarly, we have

$$\begin{aligned} \mathbb{P}(B) &\leq 1 - \mathbb{P}\left(\bigcup_{j=1}^7 E_j^{(s)}\right) \\ &= 1 + \sum_{\emptyset \neq L \subset \{1, \dots, 7\}} (-1)^{|L|} \mathbb{P}\left(X_1 \notin \bigcup_{j \in L} S_j^{(s)}\right)^p \\ &= 1 + \sum_{\emptyset \neq L \subset \{1, \dots, 7\}} (-1)^{|L|} \mathbb{P}\left(V_L^{(s)}\right)^p \end{aligned}$$

Now we compute $\mathbb{P}\left(V_L^{(s)}\right)$. Previously, we were able to “choose 255 among 256” because the S_j ’s forbid 0^8 at disjoint positions. Now the $S_j^{(s)}$ ’s forbid 0^8 at positions that may overlap. So we will decompose $\cup_{j \in L} S_j^{(s)}$ into $\cup_{C \in \mathcal{C}_L} (\cup_{j \in C} S_j)$, where each C is a *cluster* i.e. a maximal subset of

L containing consecutive elements. For example, if $L = \{1, 2, 4, 5, 6\}$, then there are two clusters $\{1, 2\}$ and $\{4, 5, 6\}$. Then

$$\mathbb{P}(V_L^{(s)}) = \prod_{C \in \mathcal{C}_L} \mathbb{P}\left(X_1 \notin \bigcup_{j \in C} S_j^{(s)}\right) = \prod_{C \in \mathcal{C}_L} \left(1 - \mathbb{P}\left(X_1 \in \bigcup_{j \in C} S_j^{(s)}\right)\right).$$

Let $R_j = [8j, 8j+7] \cup [8j-8, 8j+s-15]$ be the positions corresponding to $S_j^{(s)}$. By inclusion-exclusion again.

$$P\left(X_1 \in \bigcup_{j \in C} S_j^{(s)}\right) = \sum_{\emptyset \neq K \subset C} (-1)^{|K|+1} \mathbb{P}\left(X_1 \in \bigcap_{j \in K} S_j^{(s)}\right).$$

The event $X_1 \in \bigcap_{j \in K} S_j^{(s)}$ means that X_1 has 0's at the positions in $\bigcup_{j \in K} R_j$ and can be any string at the other positions. Hence,

$$P(V^{(s)}) = \sum_{\emptyset \neq K \subset L} (-1)^{|K|+1} \frac{1}{2^{|\bigcup_{j \in K} R_j|}}.$$

Now we have to compute $r_K = |\bigcup_{j \in K} R_j|$. We leverage *clusters* in K . Let $|K| = k \geq 1$. If K has u clusters, then

$$r_K = 8k + u(s-8).$$

The problem is reduced to counting the number of partitions of a set of size k to have u clusters. There are two stages. The first stage is to choose the sizes of the clusters i.e. $x_1, \dots, x_u \in \mathbb{N}^*$ such that

$$x_1 + \dots + x_u = k.$$

By Euler's Candies Division, there are $\binom{k-1}{u-1}$ ways to do this. The second stage is to choose the gaps between the clusters i.e. $y_0, y_u \in \mathbb{N}$ and $y_1, \dots, y_{u-1} \in \mathbb{N}^*$ such that

$$y_0 + x_1 + y_1 + \dots + y_{u-1} + x_u + y_u = 7 \Leftrightarrow y_0 + \dots + y_u = 7 - k.$$

Let $y'_0 = y_0 + 1$ and $y'_u = y_u + 1$. Then $y'_0, y_1, \dots, y_{u-1}, y'_u \in \mathbb{N}^*$ and

$$y'_0 + y_1 + \dots + y_{u-1} + y'_u = 9 - k.$$

By Euler's Candies Division again, there are $\binom{8-k}{u}$ ways to do this. Note that $\binom{8-k}{u} = 0$ if $u > k$. Therefore, the number of ways to partition a subset of size k of $\{1, \dots, 7\}$ such that there are u clusters is

$$\binom{k-1}{u-1} \binom{8-k}{u}.$$

From that formula, we derive that $u \in \min\{k, 8-k\}$. Hence,

$$\mathbb{P}\left(X_1 \in \bigcap_{j \in K} S_j^{(s)}\right) = \sum_{k=1}^7 \sum_{u=1}^{\min\{k, 8-k\}} (-1)^k \binom{k-1}{u-1} \binom{8-k}{u} \frac{1}{2^{8k+u(s-8)}}.$$

I think we can elaborate more to count the number of subsets L having v_i clusters of size i for $i \in \{1, \dots, 7\}$. I am not sure if I can get an asymptotic approximation to compare with the case without super-characters. Experiment is available in the repository, showing that the probability of a positive is significantly reduced by using super-characters. In the experiment, we iterate through subsets L of $\{1, \dots, 7\}$, then subsets K of L , no matter if S has more than one cluster or not. The formula is

$$\mathbb{P}(B) \leq 1 + \sum_{\emptyset \neq L \subset \{1, \dots, 7\}} (-1)^{|L|} \left(1 + \sum_{\emptyset \neq K \subset L} (-1)^{|K|} \frac{1}{2^{|\bigcup_{j \in K} R_j|}}\right)^p.$$

1.3 Problem Statement

In general, problem is stated as follows. Let $\mathcal{D}_P$ be the distribution of patterns and $\mathcal{D}_T$ be the distribution the text. Let the domain of the patterns be $\mathcal{X}$ and the domain of the text be $\mathcal{Y}$. We have to write the risk R_p i.e. the probability of a positive, as a function of $\mathcal{D}_P$ and $\mathcal{D}_T$ when p patterns are drawn from $\mathcal{D}_P$ and the text is drawn from $\mathcal{D}_T$.

An algorithm takes the patterns i.e. $\mathcal{X}^p$ and produces an assignment $f : \bigcup_{p \in \mathbb{N}} \mathcal{X}^p \rightarrow \{0, \dots, 7\}$.

It aims to minimize another risk $\hat{R}_p$ over only $\mathcal{X}^p$.

We would like to see if following difference can be bounded e.g.

$$\frac{1}{n} \sum_{i=1}^n \left| \min R_p - \min \hat{R}_p \right|$$

Taking Assignment Algorithm into Account

April 1st, 2026

2.1 Setting

Let P be a fixed set of patterns, T be a random text, and $B = \{0, \dots, 7\}$ be the set of buckets. An assignment function has the form $\sigma : P \rightarrow B$. For each $j \in B$ and $k \in \{0, \dots, |T| - 1\}$, let $I_{j,k}(\sigma) \in \{0, 1\}$ be a random variable such that $I_{j,k}(\sigma) = 1$ if and only if there is a positive at position k of the text, in bucket j . Let $\sigma^{-1}(j) = \{p \in P : \sigma(p) = j\}$ be the set of patterns assigned to bucket j . Let $V : 2^P \rightarrow \mathbb{R}$ be the cost of verifying if a set of patterns is a match. The cost of processing at index k of the text is

$$C_k(\sigma) = \mathbb{E} \left[\sum_{j \in B} I_{j,k}(\sigma) V(\sigma^{-1}(j)) \right].$$

An observation is that $I_{j,k}(\sigma)$ depends on the use of super-characters and strides, while $V(\sigma^{-1}(j))$ depends on hashing.

2.2 Assignment Algorithm

Firstly, we write the cost function used in the paper of Hyperscan. Let $\tilde{P} = (p_1, \dots, p_m)$ be one of a permutation of P where the patterns are sorted by length, then by lexicographical order. The assignment function is derived from a partition of $\tilde{P}$ into 8 buckets, defined by the indices $0 = b_0 < b_1 < \dots < b_6 < b_7 = m$. The patterns in bucket j are $p_{b_j}, \dots, p_{b_{j+1}-1}$. The cost function is defined as follows

$$F(b_1, \dots, b_6) = \sum_{i=0}^7 \frac{(b_{i+1} - b_i)^\alpha}{|p_{b_i}|^\beta}.$$

Proposition 1. *Define the heuristic cost function H over the set of assignment functions as*

$$H(\sigma) = \sum_{j \in B} \frac{|\sigma^{-1}(j)|^\alpha}{\min_{p \in \sigma^{-1}(j)} |p|^\beta}.$$

Then $\min_{b_1, \dots, b_6} F(b_1, \dots, b_6) = \min_{\sigma} H(\sigma)$.

Largest Concatenation-Component in Regular Expressions

April 23rd, 2026

Hyperscan implements algorithms to decompose regexes into literals and FA in its Rose engine. There may be a relation between how well a regex can be decomposed and the size of its largest concatenation-component.

3.1 Recurrences

Let $\mathcal{A}$ be a finite alphabet and $\mathcal{T}$ be the set of all regexes excluding ϵ over $\mathcal{A}$. The size function $|\cdot|$ is the number of nodes.

Definition 1 (BST-like distribution for regexes). *We define $p : \mathcal{T} \rightarrow [0, 1]$ as follows.*

1. $\sum_{c \in \mathcal{A}} p(c) = 1,$
2. $p \left(\begin{array}{c} \bullet \\ / \backslash \\ T_1 \quad T_2 \end{array} \right) = \frac{u}{n-2} p(T_1) p(T_2), \text{ where } |T_1| + |T_2| = n-1.$
3. $p \left(\begin{array}{c} + \\ / \backslash \\ T_1 \quad T_2 \end{array} \right) = \frac{v}{n-2} p(T_1) p(T_2), \text{ where } |T_1| + |T_2| = n-1.$
4. $p \left(\begin{array}{c} * \\ | \\ T \end{array} \right) = (1-u-v) p(T).$

We want to compute the expected size of the largest all-concatenation component in a regex over all of equivalent forms of its parse tree. The reason is as follows. Take the regex

$$(abc + d)(xy + z) = abctxy + abctxz + dxy + dz.$$

We expect the largest concatenation-component to be in $abctxy$ instead of abc because the former is extracted by Hyperscan. Therefore, a subtree may *yield* its concatenation-component to its parent if the parent is a concatenation node.

Recall the construction

$$\mathcal{T} = \sum_{c \in \mathcal{A}} c + \begin{array}{c} * \\ | \\ \mathcal{T} \end{array} + \begin{array}{c} \bullet \\ / \backslash \\ \mathcal{T} \quad \mathcal{T} \end{array} + \begin{array}{c} + \\ / \backslash \\ \mathcal{T} \quad \mathcal{T} \end{array}.$$

Define the random variables

1. M_n : the largest concatenation-component that is not inside a Kleene star of a regex of size n *itself*,
2. Y_n : the largest concatenation-component that is not inside a Kleene star of a regex of size n that it *yields* to its parent.

We have the following recurrences

$$Y_{n+2} = \begin{cases} Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} + 1 & \text{prob. } u, \\ \max(Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)}) & \text{prob. } v, \\ 0 & \text{prob. } 1 - u - v \end{cases} \quad Y_1 = Y_2 = 0. \quad (3.1)$$

$$M_{n+2} = \begin{cases} \max(Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} + 1, M_{U_n}^{(1)}, M_{n+1-U_n}^{(2)}) & \text{prob. } u, \\ \max(M_{U_n}^{(1)}, M_{n+1-U_n}^{(2)}) & \text{prob. } v \\ 0 & \text{prob. } 1 - u - v \end{cases} \quad M_1 = M_2 = 0. \quad (3.2)$$

We have used $Y_n^{(1)}, Y_n^{(2)}, M_n^{(1)}, M_n^{(2)}$ as independent copies of Y_n and M_n respectively, and U_n as a uniform random variable over $\{1, 2, \dots, n\}$ independent of all $Y_n^{(i)}$ and $M_n^{(i)}$.

3.2 The yield recurrence

We rewrite the recurrence (3.1) as

$$Y_{n+2} \stackrel{d}{=} I_n \left(Y_{U_n}^{(1)} + Y_{n+1-U_n}^{(2)} + 1 \right) + J_n \max \left(Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)} \right). \quad (3.3)$$

In (3.3), (I_n, J_n) is a pair of mutually exclusive Bernoulli random variables with $\mathbb{E}[I_n] = u$ and $\mathbb{E}[J_n] = v$ such that $I_n + J_n \leq 1$. The sequences $\mathbf{Y}^{(1)} = (Y_n^{(1)})_{n \geq 1}$ and $\mathbf{Y}^{(2)} = (Y_n^{(2)})_{n \geq 1}$ are independent copies of the sequence $\mathbf{Y} = (Y_n)_{n \geq 1}$. The notion of equality is in distribution. Note that we can reuse the same variables $Y_{U_n}^{(1)}, Y_{n+1-U_n}^{(2)}$ and U_n across both terms because at most one indicator is non-zero for any given realization. Let $\mathbf{I} = (I_n)_{n \geq 1}$, and define $\mathbf{J}$ and $\mathbf{U}$ similarly. We assume that the sequence of pairs $(\mathbf{I}, \mathbf{J})$ and the sequence $\mathbf{U}$ each consist of independent and identically distributed elements. Furthermore, the entire stochastic processes $(\mathbf{I}, \mathbf{J}), \mathbf{U}, \mathbf{Y}^{(1)}$, and $\mathbf{Y}^{(2)}$ are mutually independent.

3.2.1 Lower and Upper Bounds of the Expectation

Let $y_n = \mathbb{E}[Y_n]$. Take expectation on both sides, we have

$$y_{n+2} = u + \frac{2u}{n} \sum_{i=1}^n y_i + \frac{v}{n} \sum_{i=1}^n \mathbb{E}[\max(Y_i, Y_{n+1-i})].$$

For an upper bound, we use the fact that $\max(Y_i, Y_{n+1-i}) \leq Y_i + Y_{n+1-i}$, which gives an upper bound sequence

$$y_{n+2}^U = u + \frac{2(u+v)}{n} \sum_{i=1}^n y_i^U, \quad y_1^U = y_2^U = 0. \quad (3.4)$$

For a lower bound, we use

$$\begin{aligned} \mathbb{E}[\max(Y_i, Y_{n+1-i})] &= \mathbb{E} \left[\frac{1}{2} (Y_i + Y_{n+1-i} + |Y_i - Y_{n+1-i}|) \right] \\ &\geq \frac{1}{2} (\mathbb{E}[Y_i] + \mathbb{E}[Y_{n+1-i}]) + \frac{1}{2} |\mathbb{E}[Y_i] - \mathbb{E}[Y_{n+1-i}]| \\ &= \frac{1}{2} (y_i + y_{n+1-i} + |y_i - y_{n+1-i}|). \end{aligned}$$

This gives the lower bound sequence

$$y_{n+2}^L = u + \frac{2u+v}{n} \sum_{i=1}^n y_i^L + \frac{v}{2n} \sum_{i=1}^n |y_i^L - y_{n+1-i}^L|, \quad y_1^L = y_2^L = 0. \quad (3.5)$$

Remark. Another lower bound is obtained as follows. By Jensen's inequality

$$\mathbb{E}[\max(Y_i, Y_{n+1-i})] \geq \max(\mathbb{E}[Y_i], \mathbb{E}[Y_{n+1-i}]) = \max(y_i, y_{n+1-i}) = y_{\max(i, n+1-i)},$$

where the last equality holds because y_n is non-decreasing in n (*we have to prove it*). This gives the following lower bound sequence

$$y_{n+2}^L = u + \frac{2u}{n} \sum_{i=1}^n y_i^L + \frac{2v}{n} \sum_{i=\lfloor (n+1)/2 \rfloor}^n y_i^L, \quad y_1^L = y_2^L = 0. \quad (3.6)$$

Looking at the bounds (3.5), we know that when $2u + v > 1$, (y_n) grows at least polynomially.

3.2.2 Asymptotic of the Expectation

Consider the case $2u + v > 1$. By the bounds, there exists

$$\alpha = \inf \left\{ \beta : \frac{Y_n}{n^\beta} \xrightarrow{d} 0 \right\}.$$

We know that $\alpha \in (2u + v - 1, 2(u + v) - 1]$. We want to see if indeed $y_n \sim Cn^\alpha$ for some $C > 0$.

We try the contraction method [2, 3, 1]. Let us mention some conventions. For a random variable X , denote by $\mathcal{L}(X)$ its distribution. Let $(\mathcal{M}, d)$ be a metric space where $\mathcal{M}$ is a set of probability distributions of our random variables and the metric d is such that $d(\mathcal{L}(X), \mathcal{L}(Y)) = 0$ implies $X \stackrel{d}{=} Y$. The idea is as follows

1. Guess a normalization X_n of Y_n that converges in distribution.
2. Plug X_n into the recurrence. Assume that X_n converges, show that the limit equation has a unique solution X^* . This is done by proving that the operator T defined by the right-hand side of the limit equation is a contraction.
3. Prove that indeed $d(\mathcal{L}(X_n), \mathcal{L}(X^*)) \rightarrow 0$.

The method is use to obtain more information about the limit distribution of X_n rather than just its expectation. For example, the following is the recurrence for the number of comparisons in Quicksort where the first element is chosen as the pivot [1]

$$Y_n = Y_{U_n}^{(1)} + Y_{n-1-U_n}^{(2)} + n - 1.$$

One can find the asymptotic of $\mathbb{E}[Y_n]$ by taking expectation on both sides. The contraction is then used to show that $X_n = \frac{Y_n - \mathbb{E}[Y_n]}{n} \rightarrow X^*$, where the denominator n is the “guessed” variance.

Our case is weaker. The normalization is $X_n = Y_n/n^\alpha$ such that $X_n \xrightarrow{d} X^*$. Plugging into the recurrence (3.3), we have

$$\begin{aligned} (n+2)^\alpha X_{n+2} &\stackrel{d}{=} I \left(U_n^\alpha X_{U_n}^{(1)} + (n+1-U_n)^\alpha X_{n+1-U_n}^{(2)} + 1 \right) \\ &\quad + J \max \left(U_n^\alpha X_{U_n}^{(1)}, (n+1-U_n)^\alpha X_{n+1-U_n}^{(2)} \right) \end{aligned}$$

Equivalently,

$$\begin{aligned} X_{n+2} &\stackrel{d}{=} I \left(\left(\frac{U_n}{n+2} \right)^\alpha X_{U_n}^{(1)} + \left(\frac{n+1-U_n}{n+2} \right)^\alpha X_{n+1-U_n}^{(2)} + \frac{1}{(n+2)^\alpha} \right) \\ &\quad + J \max \left(\left(\frac{U_n}{n+2} \right)^\alpha X_{U_n}^{(1)}, \left(\frac{n+1-U_n}{n+2} \right)^\alpha X_{n+1-U_n}^{(2)} \right) \end{aligned} \quad (3.7)$$

Proposition 2. *The right-hand side of the above recurrence converges in distribution to*

$$I \left(U^\alpha X^{(1)} + (1 - U)^\alpha X^{(2)} \right) + J \max \left(U^\alpha X^{(1)}, (1 - U)^\alpha X^{(2)} \right) \quad (3.8)$$

Proof. We first show that

$$\left(\frac{U_n}{n+2} \right)^\alpha X_{U_n}^{(1)} + \left(\frac{n+1-U_n}{n+2} \right)^\alpha X_{n+1-U_n}^{(2)} \xrightarrow{d} U^\alpha X^{(1)} + (1 - U)^\alpha X^{(2)}.$$

By Lemma 3 and Slutsky's theorem,

$$\frac{U_n}{n+2} = \frac{U_n}{n} \cdot \frac{n}{n+2} \xrightarrow{d} U.$$

To apply the Continuous Mapping Theorem, we need to show that

$$\left(\frac{U_n}{n+2}, X_{U_n}^{(1)}, X_{n+1-U_n}^{(2)} \right) \xrightarrow{d} \left(U, X^{(1)}, X^{(2)} \right).$$

Since $U, X^{(1)}, X^{(2)}$ are independent, this is equivalent to showing that $\forall u, x_1, x_2 \in \mathbb{R}$

$$\mathbb{P} \left(U_n \leq \lfloor nu \rfloor, X_{U_n}^{(1)} \leq x_1, X_{n+1-U_n}^{(2)} \leq x_2 \right) \longrightarrow \mathbb{P}(U \leq u) \mathbb{P}(X^{(1)} \leq x_1) \mathbb{P}(X^{(2)} \leq x_2).$$

Indeed,

$$\begin{aligned} & \mathbb{P} \left(U_n \leq \lfloor nu \rfloor, X_{U_n}^{(1)} \leq x_1, X_{n+1-U_n}^{(2)} \leq x_2 \right) \\ &= \sum_{k=1}^{\lfloor nu \rfloor} \mathbb{P}(U_n = k) \mathbb{P} \left(X_k^{(1)} \leq x_1, X_{n+1-k}^{(2)} \leq x_2 | U_n = k \right) \\ &= \frac{1}{n} \sum_{k=1}^{\lfloor nu \rfloor} \mathbb{P} \left(X_k^{(1)} \leq x_1, X_{n+1-k}^{(2)} \leq x_2 \right) \quad (U_n \perp (X_k^{(1)}, X_{n+1-k}^{(2)})) \\ &= \frac{1}{n} \sum_{k=1}^{\lfloor nu \rfloor} \mathbb{P} \left(X_k^{(1)} \leq x_1 \right) \mathbb{P} \left(X_{n+1-k}^{(2)} \leq x_2 \right) \quad (X_k^{(1)} \perp X_{n+1-k}^{(2)}) \end{aligned}$$

Let F_n be the common CDF of $X_n^{(1)}$ and $X_n^{(2)}$, and F be the CDF of $X^{(1)}$. Let $m = \lfloor nu \rfloor$. The problem becomes showing that

$$\frac{1}{n} \sum_{k=1}^m F_k(x_1) F_{n+1-k}(x_2) \longrightarrow u F(x_1) F(x_2).$$

Equivalently,

$$\frac{m}{n} \left(\frac{1}{m} \sum_{k=1}^m (F_k(x_1) F_{n+1-k}(x_2) - F(x_1) F(x_2)) \right) \longrightarrow 0.$$

We have

$$\begin{aligned}
& \frac{1}{m} \sum_{k=1}^m |F_k(x_1) F_{n+1-k}(x_2) - F(x_1) F(x_2)| \\
& \leq \frac{1}{m} \sum_{k=1}^m |F_k(x_1) (F_{n+1-k}(x_2) - F(x_2))| + \frac{1}{m} \sum_{k=1}^m |F(x_2) (F_k(x_1) - F(x_1))| \\
& \leq \frac{1}{m} \sum_{k=1}^m |F_{n+1-k}(x_2) - F(x_2)| + \frac{1}{m} \sum_{k=1}^m |F_k(x_1) - F(x_1)| \quad (|F_n| \leq 1)
\end{aligned}$$

The second term goes to zero by the fact that $F_n(x_1) \rightarrow F(x_1)$ and Cesàro Means. We can write the first term as

$$\frac{1}{m} \sum_{k=n-m+1}^n |F_k(x_2) - F(x_2)|.$$

If $u = 1$, its convergence is similar to the second term. If $u < 1$, then $n - m + 1 \rightarrow \infty$. By pointwise convergence, $\forall \epsilon > 0, \exists N, \forall n \geq N : |F_n(x_2) - F(x_2)| < \epsilon$. Hence, for $n - m + 1 \geq N$,

$$\frac{1}{m} \sum_{k=n-m+1}^n |F_k(x_2) - F(x_2)| < \epsilon.$$

Using Slutsky's theorem again, we have

$$\left(\frac{n+1}{n+2}, \frac{U_n}{n+2}, X_{U_n}^{(1)}, X_{n+1-U_n}^{(2)} \right) \xrightarrow{d} \left(1, U, X^{(1)}, X^{(2)} \right).$$

Now define the function $f : \mathbb{R}_+^4 \cap \{(t, u, x, y) : t \geq u\} \rightarrow \mathbb{R}$ by

$$f(t, u, x, y) = u^\alpha x + (t - u)^\alpha y.$$

Then f is continuous. The Continuous Mapping Theorem applies. The convergence

$$\max \left(\left(\frac{U_n}{n+2} \right)^\alpha X_{U_n}^{(1)}, \left(\frac{n+1-U_n}{n+2} \right)^\alpha X_{n+1-U_n}^{(2)} \right) \xrightarrow{d} \max \left(U^\alpha X^{(1)}, (1-U)^\alpha X^{(2)} \right)$$

is similar. And finally, we define the continuous function $g(i, j, u, v) = iu + jv$ on $\mathbb{R}^4$ to finish the proof. □

Asymptotic Analysis of Some Sequences

April 28, 2026

In this section, we analyze the asymptotic behavior of lower bounds of $\mathbb{E}[Y_n]$, as follows.

$$x_{n+2} = \frac{a}{n} \sum_{i=1}^n x_i + b, \quad n \geq 1, \quad x_1 = x_2 = 0. \quad (4.1)$$

$$x_{n+2} = \frac{a}{n} \sum_{i=1}^n x_i + \frac{b}{n} \sum_{i=1}^n |x_i - x_{n+1-i}| + c, \quad n \geq 1, \quad x_1 = x_2 = 0 \quad (a > 1, 0 < b < 1, c > 0) \quad (4.2)$$

$$x_n = \frac{2u}{n} \sum_{i=0}^{n-1} x_i + \frac{2v}{n} \sum_{i=\lfloor (n-1)/2 \rfloor}^{n-1} x_i, \quad x_0 = x_1 = 1 \quad (u, v > 0, u + v \leq 1, 2u + v > 1). \quad (4.3)$$

Proposition 3. *Consider recurrence (4.1). We have*

- If $a < 1$, then $x_n \sim \frac{b}{1-a}$.
- If $a = 1$, then $x_n \sim b \ln n$.
- If $a > 1$, then $x_n \sim \frac{b}{(a-1)\Gamma(a)} n^{a-1}$.

Proof. Consider the generating function $X(z) = \sum_{n=1}^{\infty} x_n z^n$ with the convention that $x_0 = 0$. We rewrite the recurrence as

$$(n+2)x_{n+2} - 2x_{n+2} = a \sum_{i=1}^n x_i + bn.$$

We multiply both sides by z^{n+2} and sum over $n \geq 1$. The left-hand side becomes

$$\begin{aligned} \sum_{n=1}^{\infty} ((n+2)x_{n+2} - 2x_{n+2})z^{n+2} &= \sum_{n=3}^{\infty} nx_n z^n - 2 \sum_{n=3}^{\infty} x_n z^n \\ &= z \sum_{n=1}^{\infty} nx_n z^{n-1} - 2 \sum_{n=1}^{\infty} x_n z^n \\ &= zX'(z) - 2X(z). \end{aligned}$$

The right-hand side becomes

$$\begin{aligned} \sum_{n=1}^{\infty} (a \sum_{i=1}^n x_i + bn)z^{n+2} &= az^2 \sum_{n=1}^{\infty} \sum_{i=1}^n x_i z^n + bz^2 \sum_{n=1}^{\infty} nz^n \\ &= az^2 \frac{X(z)}{1-z} + \frac{bz^3}{(1-z)^2}. \end{aligned}$$

Therefore, the generating function $X(z)$ satisfies the following ODE

$$X'(z) - \left(\frac{2}{z} + \frac{az}{1-z} \right) X(z) = \frac{bz^3}{(1-z)^2}.$$

We have $\mu(z) = \exp \left(\int - \left(\frac{2}{z} + \frac{a}{1-z} - a \right) dz \right) = z^{-2}(1-z)^a$. which gives

$$z^{-2}(1-z)^a X(z) = \int b(1-z)^{a-2} dz.$$

There are two cases of a that determine the integral. If $a = 1$, we have

$$z^{-2}(1-z)X(z) = b \ln \left(\frac{1}{1-z} \right) + C.$$

Since $x_1 = x_2 = 0$, we have $z^{-2}X(z) \rightarrow 0$ as $z \rightarrow 0$, which gives $C = 0$. Therefore, we have

$$X(z) = b \frac{z^2}{1-z} \ln \left(\frac{1}{1-z} \right).$$

Hence, $x_n = bH_{n-2} \sim b \ln n$.

If $a \neq 1$, we have

$$z^{-2}(1-z)^a X(z) = -\frac{b}{a-1}(1-z)^{a-1} + C.$$

Using the same argument as above, we have $C = \frac{b}{a-1}$. Therefore, we have

$$X(z) = \frac{b}{a-1} z^2 \left((1-z)^{-a} - (1-z)^{-1} \right).$$

Using the generalized binomial theorem

$$(1-z)^{-a} = \sum_{n=0}^{\infty} \binom{n+r-1}{n} z^n,$$

we have

$$x_n = \frac{b}{a-1} \left(\binom{n-3+a}{n-2} - 1 \right) \sim \frac{b}{a-1} \left(\frac{n^{a-1}}{\Gamma(a)} - 1 \right).$$

If $a < 1$ then $x_n \sim \frac{b}{1-a}$. If $a > 1$, then $x_n \sim \frac{b}{(a-1)\Gamma(a)} n^{a-1}$. □

Lemma 1. *The sequence (x_n) in (4.2) is increasing.*

Proof. One checks that $x_n \geq 0$ for all n . Let us prove that (x_n) is increasing. We have $x_2 \geq x_1$.

Assume that $x_n \geq x_{n-1} \geq \dots \geq x_0$ for some $n \geq 3$. Then,

$$\begin{aligned}
nx_{n+2} &= a \sum_{i=1}^n x_i + b \sum_{i=1}^n |x_i - x_{n+1-i}| + nc \\
&= a \sum_{i=1}^n x_i + b \sum_{i=1}^{\lfloor n/2 \rfloor} (x_{n+1-i} - x_i) + nc \\
&= a \sum_{i=1}^n x_i + b \sum_{i=1}^{\lfloor n/2 \rfloor} (x_{n+1-i} - x_i) + nc \\
&= a \sum_{i=1}^n x_i + b \sum_{i=1}^{\lfloor n/2 \rfloor} (x_{n+1-i} - x_i) + nc \\
&= a \sum_{i=1}^n x_i + b \sum_{i=1}^{\lfloor n/2 \rfloor} (x_{n+1-i} - x_i) + nc \\
&= (a+b) \sum_{i=1}^n x_i - 2b \sum_{i=1}^{\lfloor n/2 \rfloor} x_i - bm_n + nc,
\end{aligned}$$

where

$$m_n = \begin{cases} x_{\lfloor n/2 \rfloor + 1} & \text{if } n \text{ is odd,} \\ 0 & \text{if } n \text{ is even.} \end{cases}$$

Let $S_n = \sum_{i=1}^n x_i$. Then,

$$\begin{aligned}
nx_{n+2} &= (a+b)S_n - 2bS_{\lfloor n/2 \rfloor} - bm_n + nc, \\
S_{n+1} &= S_n + x_{n+1}.
\end{aligned}$$

Hence,

$$nx_{n+2} - (n-1)x_{n+1} = (a+b)x_n - 2b(S_{\lfloor n/2 \rfloor} - S_{\lfloor (n-1)/2 \rfloor}) - b(m_n - m_{n-1}) + c.$$

Let us check the right-hand side according to the parity of n .

- If $n = 2k$, then $S_{\lfloor n/2 \rfloor} - S_{\lfloor (n-1)/2 \rfloor} = x_k$ and $m_n - m_{n-1} = -x_k$. The right-hand side is $(a+b)x_n - 2bx_k + c$.
- If $n = 2k+1$, then $S_{\lfloor n/2 \rfloor} - S_{\lfloor (n-1)/2 \rfloor} = 0$ and $m_n - m_{n-1} = x_{k+1}$. The right-hand side is $(a+b)x_n - 2bx_{k+1} + c$.

Collecting the two cases, we have

$$nx_{n+2} - (n-1)x_{n+1} = (a+b)x_n - 2bx_{\lceil n/2 \rceil} + c.$$

The proof follows from Proposition 5. □

Lemma 2. *The sequence in (4.2) can be rewritten as*

$$nx_{n+2} - (n-1)x_{n+1} = (a+b)x_n - 2bx_{\lceil n/2 \rceil} + c. \quad (4.4)$$

Proposition 4. *Consider the sequence defined by (4.3). We have $x_n = \Theta(n^\alpha)$, where α is the unique solution of*

$$2(u+v) - v2^{-\alpha} = \alpha + 1.$$

Proof. For the lower bound, assume the inductive hypothesis $x_n \geq K(n^\alpha + 1)$ with $K \leq \frac{1}{2}$ to be determined. The base cases $n = 0, 1$ hold. Let us check for $n = 2$.

$$x_2 = 2u + 2v = \alpha + 1 + v2^{-\alpha} \geq \alpha + 1 \geq \frac{1}{2}(2^\alpha + 1), \forall \alpha \in [0, 1].$$

For $n \geq 3$, we have

$$\begin{aligned} \frac{x_n}{K} &\geq \frac{2u}{n} \sum_{i=0}^{n-1} (i^\alpha + 1) + \frac{2v}{n} \sum_{i=\lfloor (n-1)/2 \rfloor}^{n-1} (i^\alpha + 1) \\ &\geq \frac{2u}{n} \left(\left(\int_0^{n-1} x^\alpha dx \right) + n \right) + \frac{2v}{n} \int_{(n-3)/2}^{n-1} (x^\alpha + 1) dx \\ &\geq \frac{2u}{n} \left(\left(\int_0^{n-1} x^\alpha dx \right) + n \right) + \frac{2v}{n} \left(\int_{(n-2)/2}^{n-1} x^\alpha + \frac{n+1}{2} \right) dx \\ &= \frac{2u}{n} \left(\frac{(n-1)^{\alpha+1}}{\alpha+1} + n \right) + \frac{2v}{n} \left(\frac{(n-1)^{\alpha+1} - (n/2)^{\alpha+1}}{\alpha+1} + \frac{n+1}{2} \right) \\ &= \frac{(n-1)^{\alpha+1}}{n(\alpha+1)} (2u + 2v - v2^{-\alpha}) + 2u + v \left(1 + \frac{1}{n} \right) \\ &\geq n^\alpha \left(1 - \frac{1}{n} \right)^{\alpha+1} + 2u + v \end{aligned}$$

We are done if we can prove that for every $n \geq 3$,

$$n^\alpha \left(1 - \frac{1}{n} \right)^{\alpha+1} + 2u + v \geq n^\alpha + 1.$$

By Bernoulli's inequality, we have $\left(1 - \frac{1}{n} \right)^{\alpha+1} \geq 1 - \frac{\alpha+1}{n}$. Hence, it suffices to show that

$$n^\alpha \left(1 - \frac{\alpha+1}{n} \right) + 2u + v \geq n^\alpha + 1 \iff 2u + v - 1 \geq \frac{\alpha+1}{n^{1-\alpha}}.$$

Let $N_0 = \left\lceil \left(\frac{\alpha+1}{2u+v-1} \right)^{\frac{1}{1-\alpha}} \right\rceil$. The inequality holds for every $n \geq N_0$. Now we choose K for it to hold for $n < N_0$. In particular,

$$K = \min \left(\frac{1}{2}, \min_{3 \leq i < N_0} \frac{x_i}{i^\alpha + 1} \right) > 0.$$

The lower bound follows. □

Induction

April 30th, 2026

Consider the sequence (x_n) defined by

$$nx_{n+2} - (n-1)x_{n+1} = (a+b)x_n - bx_{\lceil n/2 \rceil} + c, n \geq 1 \quad x_0 = x_1 = x_2 = 0, \quad (5.1)$$

where a, b and c are constants such that $a \geq 1, 0 \leq b, c \leq 1$ and $a+b \leq 2$. Suppose that $x_n \geq 0$, for all $n \geq 3$.

Note that from the recurrence (4.2), we easily have $x_n \geq 0$. If we assume that (x_n) is increasing, then we get (5.1). So we can assume $x_n \geq 0, \forall n \geq 1$ and continue to prove that (x_n) is increasing. We currently cannot prove that $x_n \geq 0$ for all $n \geq 1$ right from (5.1).

5.1 Increasingness

Proposition 5. *Consider the sequence (x_n) defined by*

$$nx_{n+2} - (n-1)x_{n+1} = (a+b)x_n - bx_{\lceil n/2 \rceil} + c, n \geq 1 \quad x_0 = x_1 = x_2 = 0, \quad (5.2)$$

where a, b and c are constants such that $a \geq 1, 0 \leq b, c \leq 1$ and $a+b \leq 2$. Suppose that $x_n \geq 0$, for all $n \geq 3$. Then (x_n) is increasing.

5.1.1 Analysis

Let $\Delta_n = x_{n+1} - x_n$, we have $\Delta_1 = 0$ and $\Delta_2 = c$. Assume that $\Delta_k \geq 0$ for all $k \leq n$. We have to show that $\Delta_{n+1} \geq 0$.

Rewrite 5 as

$$n(x_{n+2} - x_{n+1}) + x_{n+1} - x_n = (a+b-1)x_n - bx_{\lceil n/2 \rceil} + c. \quad (5.3)$$

Let $t_n = (a+b-1)x_n - bx_{\lceil n/2 \rceil} + c$. Then

$$t_{n+1} - t_n = (a+b-1)(x_{n+1} - x_n) - b(x_{\lceil (n+1)/2 \rceil} - x_{\lceil n/2 \rceil}).$$

Therefore, we have a new system

$$\begin{cases} n\Delta_{n+1} + \Delta_n = t_n, n \geq 3 \\ t_{2k} - t_{2k-1} = (a+b-1)\Delta_{2k-1}, k \geq 2 \\ t_{2k+1} - t_{2k} = (a+b-1)\Delta_{2k} - b\Delta_k, k \geq 2 \end{cases}$$

From $\Delta_{n+1} = \frac{t_n - \Delta_n}{n}$, we have an equivalence of two unproved statements

$$\Delta_{n+1} \geq 0 \iff t_n \geq \Delta_n.$$

Also from $\Delta_{n+1} = \frac{t_n - \Delta_n}{n}$ we have the equivalence of two granted statements

$$\Delta_{n-1} \geq 0 \iff \Delta_n \leq \frac{t_{n-1}}{n-1}.$$

If we can prove that $(n-1)t_n \geq t_{n-1}$, we are done. This is true for even n according to the second equation of the system. For $n = 2k+1$, we have

$$\begin{aligned} 2kt_{2k+1} - t_{2k} &= (2k-1)t_{2k+1} + (a+b-1)\Delta_{2k} - b\Delta_k \\ &\geq (2k-1)[(a+b-1)x_{2k+1} - bx_{k+1} + c] - b\Delta_k && (\Delta_{2k} \geq 0) \\ &\geq (2k-1)[(a-1)x_{2k+1} + c] - b\Delta_k && (x_{2k+1} \geq x_{k+1}) \\ &\geq (2k-1)[(a-1)(\Delta_k + c) + c] - b\Delta_k && \left(x_{2k+1} = \sum_{i=1}^{2k} \Delta_i\right) \\ &= (2k-1)ac + [(2k-1)(a-1) - b]\Delta_k \end{aligned}$$

Note that to use $x_{2k+1} \geq \Delta_k + \Delta_2 = \Delta_k + c$, we need $k \geq 3$. So we have to check the cases $n = 3, 4, 5$ separately. Now we continue.

If $a-1 \geq b$, we are done.

If $a-1 < b$. We will show that $(2k-1)ac \geq b\Delta_k$, or stronger, $\Delta_k \leq (2k-1)c$. That leads us to define a stronger inductive hypothesis

$$0 \leq \Delta_k \leq (2k-1)c, \forall 1 \leq k \leq n.$$

With that hypothesis, the lower bound is proved as before. For the upper bound, we have

$$\begin{aligned} \Delta_{n+1} &= \frac{t_n - \Delta_n}{n} \leq \frac{t_n}{n} && (\Delta_n \geq 0) \\ &= \frac{(a+b-1)x_n - bx_{\lceil n/2 \rceil} + c}{n} \leq \frac{x_n + c}{n} && (x_{\lceil n/2 \rceil} \geq 0, a+b \leq 2) \\ &= \frac{1}{n} \left(\sum_{i=1}^{n-1} \Delta_i + c \right) \\ &\leq \frac{1}{n} \left(\sum_{i=1}^{n-1} (2i-1)c + c \right) && (\Delta_i \leq (2i-1)c, \forall i \leq n) \\ &= c \frac{(n-1)^2 + 1}{n} \leq (2n+1)c. \end{aligned}$$

5.1.2 Proof

Proof. Let $\Delta_n = x_{n+1} - x_n$ and $t_n = (a+b-1)x_n - bx_{\lceil n/2 \rceil} + c$. We have

$$\begin{aligned} \Delta_1 &= 0, \Delta_2 = c, t_1 = c, t_2 = (a+b)c, \quad \text{and} \\ \begin{cases} n\Delta_{n+1} + \Delta_n = t_n, n \geq 3 \\ t_{2k} - t_{2k-1} = (a+b-1)\Delta_{2k-1}, k \geq 2 \\ t_{2k+1} - t_{2k} = (a+b-1)\Delta_{2k} - b\Delta_k, k \geq 1 \end{cases} \end{aligned} \tag{5.4}$$

Let $d = a + b$, then $1 \leq d \leq 2$. We have

$$\begin{aligned}\Delta_3 &= \frac{1}{2}(d-1)c, & t_3 &= (2d-1)c \\ \Delta_4 &= \frac{1}{6}(3d-1)c, & t_4 &= \left(\frac{1}{2}(d-1)^2 + (2d-1)\right)c \\ \Delta_5 &= \frac{1}{24}(3(d-1)^2 + 9d-5)c.\end{aligned}$$

Assume that $0 \leq \Delta_m \leq (2m-1)c$ for all $1 \leq m \leq n$. We have to show that $0 \leq \Delta_{n+1} \leq (2n+1)c$.

If $n = 2k$, we have $(2k-1)t_{2k} - t_{2k-1} \geq t_{2k} - t_{2k-1} = (a+b-1)\Delta_{2k-1} \geq 0$

$$\text{Hence } \Delta_{2k} = \frac{t_{2k-1} - \Delta_{2k-1}}{2k-1} \leq \frac{t_{2k-1}}{2k-1} \leq t_{2k}.$$

$$\text{Hence } \Delta_{2k+1} = \frac{t_{2k} - \Delta_{2k}}{2k} \geq 0.$$

If $n = 2k+1$, we have $t_{2k+1} = (a+b-1)x_{2k+1} - bx_{k+1} + c \geq (a-1)x_{2k+1} + c \geq c$.

$$\text{Hence } b\Delta_k \leq (2k-1)c \leq (2k-1)t_{2k+1}$$

$$\text{Hence } 2kt_{2k+1} - t_{2k} = (2k-1)t_{2k+1} + (a+b-1)\Delta_{2k} - b\Delta_k \geq 0.$$

$$\text{Hence } \Delta_{2k+1} = \frac{t_{2k} - \Delta_{2k}}{2k} \leq \frac{t_{2k}}{2k} \leq t_{2k+1}.$$

$$\text{Hence } \Delta_{2k+2} = \frac{t_{2k+1} - \Delta_{2k+1}}{2k+1} \geq 0.$$

The lower bound is proved. For the upper bound, we have to do exactly the same calculation as in the analysis section.

$$\begin{aligned}\Delta_{n+1} &= \frac{t_n - \Delta_n}{n} \leq \frac{t_n}{n} & (\Delta_n \geq 0) \\ &= \frac{(a+b-1)x_n - bx_{\lceil n/2 \rceil} + c}{n} \leq \frac{x_n + c}{n} & (x_{\lceil n/2 \rceil} \geq 0, a+b \leq 2) \\ &= \frac{1}{n} \left(\sum_{i=1}^{n-1} \Delta_i + c \right) \\ &\leq \frac{1}{n} \left(\sum_{i=1}^{n-1} (2i-1)c + c \right) & (\Delta_i \leq (2i-1)c, \forall i \leq n) \\ &= c \frac{(n-1)^2 + 1}{n} \leq (2n+1)c.\end{aligned}$$

□

5.2 Asymptotic

Proposition 6. *The generating function of the sequence (x_n) is given by*

$$z(1-z)X'(z) + (2z-2-(a+b)z^2)X(z) = -bz(1+z)X(z^2) + \frac{cz^3}{1-z} \quad (5.5)$$

5.2.1 Analysis

From (4.2), we can ignore the different terms and arrive at a polynomial growth. So we guess that $x_n \sim Dn^\alpha$ for some constants D and α , that is $x_n = Dn^\alpha + o(n^\alpha)$. However, x_{n+2} and x_{n+1} are multiplied by a linear term, so let us assume a stronger hypothesis $x_n = Dn^\alpha + o(n^{\alpha-1})$. Then,

$$\begin{aligned} nx_{n+2} &= n(D(n+2)^\alpha + o(n^{\alpha-1})) = Dn^{\alpha+1} \left(1 + \frac{2}{n}\right)^\alpha + O(n^\alpha) \\ &= Dn^{\alpha+1} \left(1 + \frac{2\alpha}{n} + o\left(\frac{1}{n^2}\right)\right) + o(n^\alpha) \\ &= Dn^{\alpha+1} + 2\alpha Dn^\alpha + o(n^\alpha). \end{aligned}$$

Similarly,

$$(n-1)x_{n+1} = nx_{n+1} - x_{n+1} = Dn^{\alpha+1} + (\alpha-1)Dn^\alpha + o(n^\alpha).$$

Hence,

$$nx_{n+2} - (n-1)x_{n+1} = D\alpha n^\alpha + o(n^\alpha).$$

On the other hand, we have

$$(a+b)x_n - 2bx_{\lceil n/2 \rceil} + c = D \left((a+b)n^\alpha - \frac{2b}{2^\alpha} n^\alpha \right) + o(n^\alpha).$$

So we must have

$$\alpha + 1 = a + b - b2^{-\alpha+1}.$$

Experiments support this.

Monte-Carlo Simulation for Checking Conjectures

May 04, 2026

We want to check the conjecture that the stochastic fixed-point equation

$$X \stackrel{d}{=} I(U^\alpha X^{(1)} + (1 - U)^\alpha X^{(2)}) + J \max(U^\alpha X^{(1)}, (1 - U)^\alpha X^{(2)}) \quad (6.1)$$

has a unique solution (α^*, X_{α^*}) such that $\mathbb{E}[X_{\alpha^*}] = 1$. Let T_α be the operator defined by the right-hand side of the equation, i.e.,

$$T_\alpha(\mu) = I(U^\alpha X^{(1)} + (1 - U)^\alpha X^{(2)}) + J \max(U^\alpha X^{(1)}, (1 - U)^\alpha X^{(2)}), \quad X^{(1)}, X^{(2)} \stackrel{\text{iid}}{\sim} \mu. \quad (6.2)$$

The classical contraction argument does not work. Since $X = 0$ is always a solution, we cannot use the Wasserstein metric W_p . Since T_α can change the mean of the distribution, we cannot use the Zolotarev metric ζ_p .

6.1 Background

6.1.1 Empirical Distributions

Given a distribution μ and the number of particles n , we can sample

$$X_1, X_2, \dots, X_n \stackrel{\text{iid}}{\sim} \mu.$$

The empirical distribution μ_n is

$$\mu_n = \frac{1}{n} \sum_{i=1}^n \delta_{X_i}.$$

Definition 2 (Weak convergence of distributions). *Let (M, d) be a metric space. Let $\mathcal{M}$ be the Borel σ -algebra on M . We define the distribution on $(M, \mathcal{M})$. Let $C_b(M)$ be the set of real-valued, bounded, continuous functions on M .*

A sequence of distributions (μ_n) converges weakly to a distribution μ if for $f \in C_b(M)$, we have

$$\int f d\mu_n \rightarrow \int f d\mu.$$

Theorem 1 (Varadarajan). *The sequence μ_n converges weakly to μ almost surely.*

Remark. We need to study the rate of convergence, for simulation efficiency.

By Continuous Mapping Theorem, we can do arithmetic operations on the empirical distributions.

Proposition 7. *Let $X_1, X_2, \dots, X_n \stackrel{\text{iid}}{\sim} \mu$ and $\mu_n = \frac{1}{n} \sum_{i=1}^n \delta_{X_i}$. Let $f : M \rightarrow M$ be a continuous function. Then*

$$\frac{1}{n} \sum_{i=1}^n \delta_{f(X_i)} \xrightarrow{w} f_{\#} \mu$$

almost surely.

We use the following particular cases

1. When $f(x, y) = x + y$, then $\frac{1}{n} \sum_{i=1}^n \delta_{X_i+Y_i} \xrightarrow{w} \mathcal{L}(X + Y)$ almost surely.
2. When $f(x, y) = \max(x, y)$, then $\frac{1}{n} \sum_{i=1}^n \delta_{\max(X_i, Y_i)} \xrightarrow{w} \mathcal{L}(\max(X, Y))$ almost surely.
3. when $f(x, y) = xy$, then $\frac{1}{n} \sum_{i=1}^n \delta_{X_i Y_i} \xrightarrow{w} \mathcal{L}(XY)$ almost surely.

Proposition 8. *Let (μ_n) be empirical measures such that $(\mu_n) \xrightarrow{w} \mu$ almost surely and the p -th moment of μ is finite. Then*

$$W_p(\mu_n, \mu) \rightarrow 0$$

almost surely.

Using the triangle inequality, we can show the following convergence result.

Proposition 9 (Convergence of Wasserstein distance). *Let $(\mu_n) \xrightarrow{w} \mu$ almost surely and $\nu_n \xrightarrow{w} \nu$ almost surely. Then*

$$W_p(\mu_n, \nu_n) \rightarrow W_p(\mu, \nu)$$

almost surely.

Proposition 9 provides an $O(n \log n)$ algorithm to compute the empirical Wasserstein distance

$$W_p(\mu_n, \nu_n) = \frac{1}{n} \inf_{i,j} \left(\sum_{i=1}^n |X_i - Y_j|^p \right)^{1/p} = \frac{1}{n} \left(\sum_{i=1}^n |\hat{X}_i - \hat{Y}_i|^p \right)^{1/p}, \quad (6.3)$$

where $(\hat{X}_1, \dots, \hat{X}_n)$ and $(\hat{Y}_1, \dots, \hat{Y}_n)$ are the sorted samples from μ_n and ν_n , respectively.

6.1.2 Random Distributions

Our conjectures include convergence of distributions. We need a way to generate random distributions. In particular, we need to generate random distributions with mean 1. A straightforward way is to use a bank of parametric distributions, such as uniform, log-normal, and gamma distributions. The base distribution is chosen randomly from the bank, and the shape parameters are also randomized. Finally, we can normalize the distribution to have mean 1.

A problem with the last approach is expressivity. Let us study Dirichlet processes.

Appendix

Lemma 3. *Let U_n be a uniform random variable on $\{1, 2, \dots, n\}$ and let U be a uniform random variable on $[0, 1]$. Then $\frac{U_n}{n} \xrightarrow{d} U$.*

Proof. Recall the CDF of U

$$F_U(x) = \begin{cases} 0 & x < 0, \\ x & 0 \leq x \leq 1, \\ 1 & x > 1. \end{cases}$$

Let $V_n = U_n/n$. We have $F_{V_n}(x) = \mathbb{P}(U_n/n \leq x) = \mathbb{P}(U_n \leq nx)$. Hence

- If $x \leq 0$, then $F_{V_n}(x) = 0 = F_U(x)$, since $U_n \geq 1$.
- If $x > 1$, then $F_{V_n}(x) = 1 = F_U(x)$, since $U_n \leq n$.
- If $0 < x \leq 1$, then $F_{V_n}(x) = \frac{\lfloor nx \rfloor}{n}$. Since $x - \frac{1}{n} \leq \frac{\lfloor nx \rfloor}{n} \leq x$, we have $F_{V_n}(x) \rightarrow x = F_U(x)$.

□

Lemma 4. *Let (X_n) be a sequence of random variables converging in distribution to X . Let (N_n) be a integer-valued sequence of random variables independent of (X_n) such that $N_n \xrightarrow{d} \infty$. Then $X_{N_n} \xrightarrow{d} X$.*

Proof. We want to show that $\lim_{n \rightarrow \infty} \mathbb{P}(X_{N_n} \leq x) = \mathbb{P}(X \leq x)$ for all x at which F_X is continuous. The CDF of X_{N_n} is

$$\mathbb{P}(X_{N_n} \leq x) = \sum_{k=1}^{\infty} \mathbb{P}(X_k \leq x, N_n = k) = \sum_{k=1}^{\infty} \mathbb{P}(N_n = k) \mathbb{P}(X_k \leq x),$$

where the last equality holds because X_k and N_n are independent. Let $\epsilon > 0$. Since $X_n \xrightarrow{d} X$, there exists K such that for all $k \geq K$, we have

$$|\mathbb{P}(X_k \leq x) - \mathbb{P}(X \leq x)| < \frac{\epsilon}{2}.$$

Since $\sum_{k=1}^{\infty} \mathbb{P}(N_n = k) = 1$, we have

$$\begin{aligned}
|\mathbb{P}(X_{N_n} \leq x) - \mathbb{P}(X \leq x)| &= \left| \sum_{k=1}^{\infty} \mathbb{P}(N_n = k) (\mathbb{P}(X_k \leq k) - \mathbb{P}(X \leq x)) \right| \\
&\leq \sum_{k=1}^{\infty} \mathbb{P}(N_n = k) |\mathbb{P}(X_k \leq k) - \mathbb{P}(X \leq x)| \\
&\leq \sum_{k=1}^{K-1} \mathbb{P}(N_n = k) |\mathbb{P}(X_k \leq k) - \mathbb{P}(X \leq x)| \\
&\quad + \sum_{k=K}^{\infty} \mathbb{P}(N_n = k) |\mathbb{P}(X_k \leq k) - \mathbb{P}(X \leq x)| \\
&\leq \sum_{k=1}^{K-1} \mathbb{P}(N_n = k) \cdot 1 + \sum_{k=K}^{\infty} \mathbb{P}(N_n = k) \cdot \frac{\epsilon}{2}
\end{aligned}$$

Since $N_n \xrightarrow{d} \infty$, we have $\mathbb{P}(N_n \geq K) \rightarrow 1$ as $n \rightarrow \infty$. Hence, there exists N such that for all $n \geq N$, we have $\mathbb{P}(N_n \geq K) > 1 - \frac{\epsilon}{2}$. Therefore, for all $n \geq N$, we have

$$|\mathbb{P}(X_{N_n} \leq x) - \mathbb{P}(X \leq x)| < \frac{\epsilon}{2} + \frac{\epsilon}{2} \left(1 - \frac{\epsilon}{2}\right) \leq \epsilon.$$

Since ϵ is arbitrary, the result follows. □

References

- [1] Michael Drmota. *Random trees: an interplay between combinatorics and probability*. Springer, 2009.
- [2] R Ludger. “The contraction method for recursive algorithms”. In: ().
- [3] Ralph Neininger and Ludger Rüschendorf. “A general limit theorem for recursive algorithms and combinatorial structures”. In: *The Annals of Applied Probability* 14.1 (2004), pp. 378–418.
- [4] Xiang Wang et al. “Hyperscan: A fast multi-pattern regex matcher for modern {CPUs}”. In: *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. 2019, pp. 631–648.